

Estrutura Sequencial

Email

Prof.s3rgio@gmail.com

SINTAXE

```
#include <nome_da_biblioteca>
int main()
{
    bloco_de_comandos;
    return 0;
}
```

```
#include <nome_da_biblioteca>
int main()
{
    bloco_de_comandos;
    return 0;
}
```

Bibliotecas são arquivos contendo várias funções que podem ser incorporadas aos programas escritos em C/C++.

A diretiva **#include** faz o texto contido na biblioteca especificada ser inserido no programa.

A biblioteca **stdio.h** permite a utilização de diversos comandos de entrada e saída.

```
#include <nome_da_biblioteca>
int main()
{
    bloco_de_comandos;
    return 0;
}
```

É importante salientar que a linguagem C/C++ é **sensível a letras maiúsculas e minúsculas**, ou seja, considera que letras maiúsculas são diferentes de minúsculas.

Sendo assim (por exemplo, **a** é diferente de **A**), **todos os comandos devem, obrigatoriamente, ser escritos com letras minúsculas.**

DECLARAÇÃO DE VARIÁVEIS

As variáveis são declaradas após a especificação de seus tipos.

Os tipos de dados mais utilizados são:

- **int** (para números inteiros);
- **float** (para números reais) e
- **char** (para um caractere).

A linguagem C/C++ **não possui tipo de dados boolean** (que pode assumir os valores verdadeiro ou falso), pois considera verdadeiro qualquer valor diferente de 0 (**zero**).

A linguagem C **não possui um tipo especial para armazenar cadeias de caracteres (strings).**

Deve-se, quando necessário, utilizar um vetor contendo vários elementos do tipo char.

Os vetores serão tratados em aulas futuras.

EXEMPLOS DE DECLARAÇÃO

```
float X;
```

Declara uma **variável chamada X** em que pode ser armazenado um número real.

```
float Y, Z;
```

Declara duas **variáveis chamada Y e Z** em que podem ser armazenados dois números reais.

```
char SEXO;
```

Declara uma **variável chamada SEXO** em que pode ser armazenado um CARACTERE.

```
char NOME[40];
```

Declara uma **variável chamada NOME** em que podem ser armazenados até 39 caracteres. O 40º caractere será o \0, que indica final de cadeia de caracteres.

A linguagem C/C++ possui **quatro tipos básicos** que podem ser utilizados na declaração das variáveis: **int**, **float**, **double** e **char**.

A partir desses tipos básicos, podem ser definidos outros, conforme apresentado na tabela a seguir.

CONCEITO VARIÁVEIS

Tipo	Faixa de valores	Tamanho (aproximado)
char	-128 a 127	8 bits
unsigned char	0 a 255	8 bits
int	-32.768 a 32.767	16 bits
unsigned int	0 a 65.535	16 bits
short int	-32.768 a 32.767	16 bits
long	-2.147.483.648 a 2.147.483.647	32 bits
unsigned long	0 a 4.294.967.295	32 bits
float	3.4×10^{-38} a 3.4×10^{38}	32 bits
double	1.7×10^{-308} a 1.7×10^{308}	64 bits
long double	3.4×10^{-4932} a 1.1×10^{4932}	80 bits

É importante ressaltar que, de acordo com o processador ou compilador C/C++ utilizado, o tamanho e a faixa de valores podem variar. As faixas apresentadas seguem o padrão ANSI e são consideradas mínimas.

DECLARAÇÃO DE CONSTANTES

As constantes são declaradas depois das bibliotecas e seus valores não podem ser alterados durante a execução do programa.

A declaração de uma constante deve obedecer à seguinte sintaxe:

```
#define nome valor
```

EXEMPLOS DE DECLARAÇÃO

```
#define x 7
```

Define uma constante com **identificador x** e **valor 7**.

```
#define y 4.5
```

Define uma constante com **identificador y** e **valor 4.5**.

```
#define nome "MARIA"
```

Define uma constante com **identificador nome** e **valor MARIA**.

COMANDO DE ATRIBUIÇÃO

O comando de **atribuição** é utilizado para **conceder valores ou operações a variáveis**, sendo **representado por =** (sinal de igualdade).

EXEMPLO

```
x = 4;  
x = x + 2;  
y = 2.5;  
sexo = 'F';
```

Caso seja necessário armazenar uma cadeia de caracteres dentro de uma variável, deve-se utilizar uma função para manipulação de caracteres, conforme apresentado a seguir:

```
strcpy(nome, "João");
```

```
strcpy (nome, "João") ;
```

Para que seja possível a utilização da função **strcpy**, deve-se inserir no programa, por meio da diretiva **include**, a biblioteca ***string.h***.

As funções de manipulação de ***strings*** serão abordadas em aulas futuras.

OBSERVAÇÕES



Em C/C++, os caracteres são representados entre **apóstrofos** (').

As cadeias de caracteres devem ser representadas entre **aspas** (").

Em C/C++, **cada comando é finalizado com o sinal de ponto e vírgula.**

Em C/C++, **a parte inteira e a parte fracionária do número são separadas por um ponto.**



COMANDO DE ENTRADA

O comando de entrada é utilizado para receber dados digitados pelo usuário.

Os dados recebidos **são armazenados em variáveis**.

Um dos **comandos de entrada** mais utilizados na linguagem C/C++ é o ***scanf***.

EXEMPLO

```
scanf ("%d%c", &X) ;
```

Um **valor inteiro**, digitado pelo usuário, será **armazenado na variável X**.

```
scanf ("%f%c", &Z) ;
```

Um **valor real**, digitado pelo usuário, será **armazenado na variável Z**.

EXEMPLO

```
scanf ("%s%c", &NOME) ;
```

Um ou mais caracteres, digitados pelo usuário, serão armazenados na **variável NOME**.

```
scanf ("%c%c", &Y) ;
```

Um caractere, digitado pelo usuário, será armazenado na **variável Y**.

No comando ***scanf***, é necessário indicar o tipo de variável que será lida:

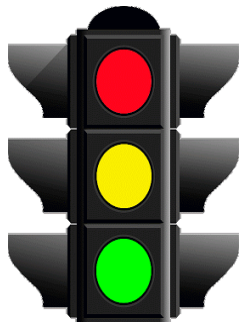
%f para variáveis que armazenam **números reais**;

%d para variáveis que armazenam **números inteiros**;

%c para variáveis que armazenam **um único caractere** e

%s para variáveis que armazenam **um conjunto de caracteres**.

O comando **scanf** armazena em um buffer o conteúdo digitado pelo usuário e armazena também a tecla enter utilizada pelo usuário para encerrar a entrada de dados.



Para que o buffer **seja esvaziado** depois da atribuição do conteúdo à variável, **utiliza-se %*c.**

COMANDO DE SAÍDA

O comando de **saída** é utilizado para mostrar dados na tela ou na impressora.

. Um dos comandos de saída mais utilizado na linguagem C/C++ é o ***printf***.

EXEMPLO

```
printf("%d", Y) ;
```

Mostra o **número inteiro** armazenado na **variável Y**.

```
printf("Conteúdo de Y = %d", Y) ;
```

Mostra a mensagem "**Conteúdo de Y =** " e, em seguida, o **número inteiro** armazenado na **variável Y**.

EXEMPLO

```
printf("%f", x);
```

Mostra o **número real** armazenado na **variável X**.

```
printf("%5.2f", x);
```

Mostra o **número real** armazenado na **variável X** utilizando cinco caracteres da tela, e, destes, **dois serão utilizados para a parte fracionária** e um para o ponto, que é o separador da parte inteira e da parte fracionária.

EXEMPLO

```
printf("Conteúdo de X = %7.3f", x);
```

Mostra a mensagem “**Conteúdo de X =** ” e, em seguida, o **número real** armazenado na **variável X**, **utilizando sete caracteres da tela**, e, destes, **três serão utilizados para a parte fracionária** e um para o ponto, que é o separador da parte inteira e da parte fracionária.

No comando *printf*, é necessário indicar o tipo de variável que será **mostrada**:

%f para variáveis que armazenam **números reais**;

%d para variáveis que armazenam **números inteiros**;

%c para variáveis que armazenam **um único caractere** e

%s para variáveis que armazenam **um conjunto de caracteres**.

No comando ***printf*** pode-se utilizar caracteres para posicionar a saída, por exemplo, **\n**, **que passa o cursor para a próxima linha**, ou **\t**, **que avança o cursor uma tabulação**.

Exemplo 01 :

```
printf("aula ");  
printf("fácil");
```

Os comandos acima geram a saída a seguir:

aula fácil

Exemplo 02 :

```
printf("aula ");  
printf("\nfácil");
```

Os comandos acima geram a saída a seguir:

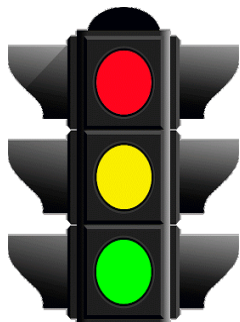
```
aula  
fácil
```

Exemplo 03 :

```
printf("aula ");  
printf("\tfácil");
```

Os comandos acima geram a saída a seguir:

Aula	fácil
-------------	--------------



COMENTÁRIOS

em

C/C++

Comentários são textos que podem ser inseridos em programas com o objetivo de documentá-los.

Eles não são analisados pelo compilador.

Os comentários podem ocupar uma ou várias linhas, devendo ser inseridos nos programas utilizando-se os símbolos `/* ..... */` ou `//`

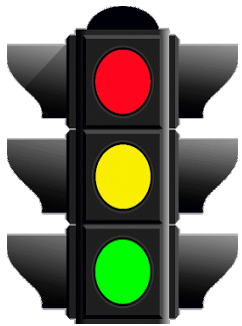
EXEMPLO

```
/*  
linhas de comentário  
linhas de comentário  
*/
```

A região de comentários é aberta com os símbolos **/*** e encerrada com os símbolos ***/**.

```
// comentário
```

A região de comentários é aberta com os símbolos **//** e encerrada automaticamente ao final da linha.



OPERADORES E FUNÇÕES PREDEFINIDAS

A linguagem **C/C++** possui **operadores** e **funções** predefinidas destinados a cálculos matemáticos.
Alguns são apresentados a seguir.

Operador	Exemplo	Comentário
=	$x = y$	O conteúdo da variável Y é atribuído à variável X (A uma variável pode ser atribuído o conteúdo de outra, um valor constante ou, ainda, o resultado de uma função).
+	$x + y$	Soma o conteúdo de X e de Y.
-	$x - y$	Subtrai o conteúdo de Y do conteúdo de X.
*	$x * y$	Multiplica o conteúdo de X pelo conteúdo de Y.
/	x / y	Obtém o quociente da divisão de X por Y. Se os operandos são inteiros, o resultado da operação será o quociente inteiro da divisão. Se os operandos são reais, o resultado da operação será a divisão. Por exemplo: $\text{int } z = 5/2; \rightarrow$ a variável z receberá o valor 2. $\text{float } z = 5.0/2.0; \rightarrow$ a variável z receberá o valor 2.5.
%	$x \% y$	Obtém o resto da divisão de X por Y.

O operador **%** só pode ser utilizado **com operandos do tipo inteiro**.

Operador	Exemplo	Comentário
<code>+=</code>	<code>x += y</code>	Equivale a <code>X = X + Y</code> .
<code>--=</code>	<code>x -= y</code>	Equivale a <code>X = X - Y</code> .
<code>*=</code>	<code>x *= y</code>	Equivale a <code>X = X * Y</code> .
<code>/=</code>	<code>x /= y</code>	Equivale a <code>X = X / Y</code> .
<code>%=</code>	<code>x %= y</code>	Equivale a <code>X = X % Y</code> .
<code>++</code>	<code>x++</code>	Equivale a <code>X = X + 1</code> .
<code>++</code>	<code>y = ++x</code>	Equivale a <code>X = X + 1</code> e depois <code>Y = X</code> .
<code>++</code>	<code>y = x++</code>	Equivale a <code>Y = X</code> e depois <code>X = X + 1</code> .
<code>--</code>	<code>x--</code>	Equivale a <code>X = X - 1</code> .
<code>--</code>	<code>y = --x</code>	Equivale a <code>X = X - 1</code> e depois <code>Y = X</code> .
<code>--</code>	<code>y = x--</code>	Equivale a <code>Y = X</code> e depois <code>X = X - 1</code> .

Os operadores matemáticos de atribuição são utilizados para representar de maneira sintética uma operação aritmética e, posteriormente, uma operação de atribuição.

Operador	Exemplo	Comentário
<code>+=</code>	<code>x += y</code>	Equivale a <code>X = X + Y</code> .

Por exemplo, na tabela acima o operador **+=** está sendo usado para realizar a operação **x + y** e, posteriormente, **atribuir o resultado obtido à variável x**.

OPERADORES

Operador	Exemplo	Comentário
<code>==</code>	<code>x == y</code>	O conteúdo de X é igual ao conteúdo de Y.
<code>!=</code>	<code>x != y</code>	O conteúdo de X é diferente do conteúdo de Y.
<code><=</code>	<code>x <= y</code>	O conteúdo de X é menor ou igual ao conteúdo de Y.
<code>>=</code>	<code>x >= y</code>	O conteúdo de X é maior ou igual ao conteúdo de Y.
<code><</code>	<code>x < y</code>	O conteúdo de X é menor que o conteúdo de Y.
<code>></code>	<code>x > y</code>	O conteúdo de X é maior que o conteúdo de Y.

Funções matemáticas		
Função	Exemplo	Comentário
<code>ceil</code>	<code>ceil(X)</code>	Arredonda um número real para cima. Por exemplo, <code>ceil(3.2)</code> é 4.
<code>cos</code>	<code>cos(X)</code>	Calcula o cosseno de X (X deve estar representado em radianos).
<code>exp</code>	<code>exp(X)</code>	Obtém o logaritmo natural e elevado à potência X.
<code>abs</code>	<code>abs(X)</code>	Obtém o valor absoluto de X.
<code>floor</code>	<code>floor(X)</code>	Arredonda um número real para baixo. Por exemplo, <code>floor(3.2)</code> é 3.
<code>log</code>	<code>log(X)</code>	Obtém o logaritmo natural de X.
<code>log10</code>	<code>log10(X)</code>	Obtém o logaritmo de base 10 de X.
<code>modf</code>	<code>z = modf(X, &Y)</code>	Decompõe o número real armazenado em X em duas partes: Y recebe a parte fracionária e z, a parte inteira do número.
<code>pow</code>	<code>pow(X, Y)</code>	Calcula a potência de X elevado a Y.
<code>sin</code>	<code>sin(X)</code>	Calcula o seno de X (X deve estar representado em radianos).
<code>sqrt</code>	<code>sqrt(X)</code>	Calcula a raiz quadrada de X.
<code>tan</code>	<code>tan(X)</code>	Calcula a tangente de X (X deve estar representado em radianos).

PALAVRAS RESERVADAS

Palavras reservadas são nomes utilizados pelo compilador para representar comandos de controle do programa, operadores e diretivas. As palavras reservadas da linguagem C/C++ são:

asm	auto	break	case	cdecl	char
class	const	continue	_cs	default	delete
do	double	_ds	else	enum	_es
export	extern	far	_fastcall	float	friend
goto	huge	for	if	inline	int
interrupt	_loadds	long	near	new	operator
pascal	private	protected	public	register	return
_saverregs	_seg	short	signed	sizeof	_ss
static	struct	switch	template	this	typedef
union	unsigned	virtual	void	volatile	while

EXERCÍCIOS PRÁTICOS

EXERCICIO 01

1. Faça um programa que **receba o salário de um funcionário, calcule e mostre:**
 - o salário atual,
 - o valor do aumento e
 - o novo salário, sabendo-se que este sofreu um aumento de 25%.

SOLUÇÃO EXE 01_A

SOLUÇÃO EXE 01_B

EXERCICIO 02

Faça um programa que **receba o salário base de um funcionário**, **calcule e mostre o salário a receber**, sabendo-se que o funcionário tem **gratificação de 5% sobre o salário base** e **paga imposto de 7% também sobre o salário base**.

Faça **exibir o valor da gratificação, valor do imposto e o salário a receber**.

SOLUÇÃO EXE 02

EXERCICIO 03

Faça um programa que **receba o valor de um depósito** e o **valor da taxa de juros**, **calcule e mostre o valor do rendimento e o valor total depois do rendimento.**

SOLUÇÃO EXE 03

EXERCICIO 04

Sabe-se que: **pé = 12 polegadas**, **1 jarda = 3 pés**, **1 milha = 1760 jardas**.

Faça um programa que **receba uma medida em pés**, **faça as conversões a seguir e mostre os resultados**.

- a) polegadas;
- b) jardas;
- c) milhas.

SOLUÇÃO EXE 04

FIM !

REFERENCIAS

Fundamentos da Programação de Computadores
– 3ª Edição , ASCENCIO, Ana Fernanda G. e
CAMPOS, Edilene Ap. V. – Editora Person

Como Programar C - 6ª Edição , DEITEL, Paul e
DEITEL, Harvey – Editora Person